



Bug Bounty Report

Integer Underflow in `_credit()` Permanently Locks
Bridged Funds on Spoke Chains

alva-p.xyz

April 2026

Prepared by: alva-p Lead Researcher: Pineda Alvaro

Table of Contents

- Summary
 - Root Cause
 - How the Underflow Propagates Through Bridging
 - The Deadlock Condition
 - Happy Path vs Attack Path
 - Proof of Concept
 - Fix Direction
 - Honest Report Boundary
 - Timeline
-

Summary

The `_credit()` function in `OverlayerWrapCore` unconditionally decrements `totalBridgedOut` on every chain where the contract is deployed. On spoke chains, `totalBridgedOut` is initialized to 0 and is never incremented – there are no outbound operations originating from spoke chains before the first inbound bridge credit arrives.

When the hub chain sends `OverlayerWrap` tokens to a spoke via LayerZero OFT, the spoke's `_credit()` executes `0 - amountReceivedLD`. Solidity 0.8.x checked arithmetic reverts on underflow, causing the entire `lzReceive` transaction to fail. LayerZero message retries reproduce the same revert deterministically. Tokens burned on the hub are never minted on the spoke – the loss is permanent.

Severity: Critical (Impact: High, Likelihood: High) **Impact:** All hub-to-spoke bridging is permanently non-operational; 100% of bridged value is at risk **Status:** Confirmed and fixed

Root Cause

File: `contracts/overlayer/OverlayerWrapCore.sol:537-544`

```
1 function _credit(  
2     address to_,  
3     uint256 amountLD_,  
4     uint32 srcEid_  
5 ) internal virtual override returns (uint256 amountReceivedLD) {  
6     amountReceivedLD = super._credit(to_, amountLD_, srcEid_);  
7 -   totalBridgedOut -= amountReceivedLD; // UNDERFLOW on spoke  
    chains  
8 }
```

`totalBridgedOut` is a hub-chain accounting variable. Its correct semantics are:

- **Hub `_debit` (outbound):** `totalBridgedOut += amountSentLD` – tokens leave the hub
- **Hub `_credit` (inbound return):** `totalBridgedOut -= amountReceivedLD` – tokens return to hub

The `_credit()` override is deployed identically on all chains with no chain-identity guard. On spoke chains, `totalBridgedOut` is always 0. The decrement `0 - amountReceivedLD` triggers Solidity's checked arithmetic and reverts.

Result: `_credit()` always reverts on any spoke chain receiving its first inbound message.

How the Underflow Propagates Through Bridging

The call chain from LayerZero delivery to the failed state change:

```
1 lzReceive() [LayerZero Endpoint -- Arbitrum]  
2 |  
3 +--> OFTCore._lzReceive()  
4 |  
5 +--> OverlayerWrapCore._credit(to, 1000e18, srcEid)  
6 |  
7 +--> super._credit() // would mint 1000e18 to  
    recipient  
8 |  
9 +--> totalBridgedOut -= 1000e18  
10 // spoke state: totalBridgedOut == 0  
11 // 0 - 1000e18 => ARITHMETIC UNDERFLOW =>  
    REVERT
```

The entire `lzReceive` transaction reverts. No tokens are minted. LayerZero stores the failed message for retry, but every retry hits the same revert since no state has changed.

The Deadlock Condition

One might ask: could a user on the spoke first bridge *out* (incrementing `totalBridgedOut`) before receiving tokens inbound? This is impossible:

- To hold spoke-chain `OverlayerWrap`, a user must receive it via a successful `_credit()`
- `_credit()` always reverts on the spoke
- There is no alternative minting path

The system has no valid initial state from which spoke chains can become operational. The deadlock is absolute.

Happy Path vs Attack Path

Scenario	Chain	<code>totalBridgedOut</code>		
			Operation	Result
Hub send (outbound)	Hub	0	<code>+= 1000e18</code>	1000e18 – OK
Hub receive (return)	Hub	1000e18	<code>-= 1000e18</code>	0 – OK
Spoke receive (first)	Spoke	0	<code>-= 1000e18</code>	REVERT
Spoke receive (after fix)	Spoke	0	skipped (not hub)	mint succeeds – OK

The hub chain operates correctly in both directions. Only spoke-chain inbound credits are broken.

Proof of Concept

The following self-contained Foundry test reproduces the underflow without external dependencies.

Target: <https://github.com/Overlayerfi/contracts/tree/519c9e92fd9d80d11e35e9868130f6334b88d6>

```
1 // SPDX-License-Identifier: GPL-3.0
2 pragma solidity ^0.8.20;
3
4 import "forge-std/Test.sol";
5
6 contract CreditUnderflowTest is Test {
7     uint256 public totalBridgedOut;
8
9     function test_creditUnderflowOnSpokeChain() public {
10         // Spoke chain state: totalBridgedOut = 0 (never debited)
11         assertEq(totalBridgedOut, 0);
12
13         // Simulate _credit being called when hub sends tokens to
14         // spoke
15         uint256 amountReceivedLD = 1000e18;
16
17         // Reverts: arithmetic underflow in Solidity 0.8.20
18         vm.expectRevert();
19         totalBridgedOut -= amountReceivedLD;
20     }
21
22     function test_hubDebitWorksButSpokeCreditFails() public {
23         // Hub chain: _debit works correctly
24         uint256 amountSentLD = 1000e18;
25         totalBridgedOut += amountSentLD; // 0 + 1000e18 = 1000e18
26         -- OK
27
28         // Simulate spoke chain (separate deployment,
29         // totalBridgedOut = 0)
30         uint256 spokeTotalBridgedOut = 0;
31         uint256 amountReceivedLD = 1000e18;
32
33         // 0 - 1000e18 underflows
34         vm.expectRevert();
35         spokeTotalBridgedOut -= amountReceivedLD;
36     }
37 }
```

Run with:

```
1 forge test --match-contract CreditUnderflowTest -vvvv
```

Expected output (2/2 pass):

```
1 [PASS] test_creditUnderflowOnSpokeChain()
2     spoke totalBridgedOut = 0
3     0 - 1000e18 => arithmetic underflow => REVERT confirmed
4
```

```
5 [PASS] test_hubDebitWorksButSpokeCreditFails()
6   hub: 0 + 1000e18 = 1000e18 -- OK
7   spoke: 0 - 1000e18 => arithmetic underflow => REVERT confirmed
8
9 2 tests passed
```

On-chain reproduction steps:

1. Deploy `OverlayerWrap.sol` on two chains (Ethereum as hub, Arbitrum as spoke) with the same contract code, setting `hubChainId` to Ethereum's chain ID on both deployments
2. On the hub: mint `OverlayerWrap` by depositing USDC collateral, call `send()` to bridge tokens to Arbitrum
3. Observe: `_debit()` executes successfully on hub, burns tokens, increments `totalBridgedOut`
4. On Arbitrum: LayerZero delivers the cross-chain message, `_credit()` is called internally by `lzReceive()` – `totalBridgedOut -= amountReceivedLD` executes with `totalBridgedOut == 0`
5. Result: underflow revert, tokens permanently destroyed

Fix Direction

Primary Fix – `OverlayerWrapCore.sol`

Guard `totalBridgedOut` mutations behind a hub chain identity check. Both `_debit` and `_credit` must be patched – fixing only `_credit` stops the revert but leaves `_debit` writing incorrect state on spoke chains for any outbound transfers.

```
1   function _credit(
2       address to_,
3       uint256 amountLD_,
4       uint32 srcEid_
5   ) internal virtual override returns (uint256 amountReceivedLD) {
6       amountReceivedLD = super._credit(to_, amountLD_, srcEid_);
7 -   totalBridgedOut -= amountReceivedLD;
8 +   if (block.chainid == hubChainId) {
9 +       totalBridgedOut -= amountReceivedLD;
10  +   }
11  }
12
13  function _debit(
14      address from_,
```

```
15     uint256 amountLD_,
16     uint256 minAmountLD_,
17     uint32 dstEid_
18 )
19     internal
20     virtual
21     override
22     returns (uint256 amountSentLD, uint256 amountReceivedLD)
23 {
24     (amountSentLD, amountReceivedLD) = super._debit(
25         from_, amountLD_, minAmountLD_, dstEid_
26     );
27 -     totalBridgedOut += amountSentLD;
28 +     if (block.chainid == hubChainId) {
29 +         totalBridgedOut += amountSentLD;
30 +     }
31 }
```

Defense in Depth

Add an explicit underflow guard in `_credit` to prevent silent fund loss from any future chain-identity misconfiguration:

```
1 if (totalBridgedOut >= amountReceivedLD) {
2     totalBridgedOut -= amountReceivedLD;
3 }
```

Deployment Note

Any users whose bridge transactions failed due to this bug had tokens burned on the hub with no spoke mint. A guardian-controlled re-mint or refund mechanism on the hub should be evaluated as part of the fix rollout.

Honest Report Boundary

What Is Directly Proven

- `_credit()` unconditionally decrements `totalBridgedOut` on all chain deployments, including spokes where the value is always 0

-
- 0 - `amountReceivedLD` triggers Solidity 0.8.x checked arithmetic and reverts (verified by PoC)
 - The deadlock is absolute: there is no code path by which `totalBridgedOut` on a spoke can become non-zero before the first `_credit()` call
 - The revert is deterministic – LayerZero message retries will reproduce the same failure without state change
 - Both `_debit` and `_credit` must be patched; a partial fix leaves corrupted spoke-chain accounting
 - The 1-line hub guard (shown in the diff above) fully restores correct behavior on both hub and spoke chains

What Is Modeled but Not End-to-End

Live cross-chain execution not demonstrated: The PoC reproduces the underflow in isolation using Foundry. A full two-chain deployment against live LayerZero endpoints was not executed. The revert path follows directly from the code, but mainnet fund loss was not triggered.

Recovery mechanism not assessed: The protocol does not expose a recovery path for tokens burned on the hub under a failed bridge. The impact (permanent fund loss) follows from the absence of any such mechanism in the reviewed commit.

Timeline

Date	Event
April 9, 2026	Report submitted
April 14, 2026	Bug validated as Critical by triage team
April 14, 2026	Triage note: <code>_credit()</code> underflow on non-hub chains confirmed
April 19, 2026	Final results announced – Critical severity
April 19, 2026	Bounty paid
April 19, 2026	Permission requested to publish this report
